Examen parcial de Manejo y Estructura de Datos.

Indicaciones generales

- Lea cuidadosamente cada enunciado antes de responder.
- Justifique sus decisiones técnicas; las definiciones sin relación con el caso pueden recibir puntaje parcial.
- En las preguntas de trazado, muestre el estado de la estructura después de cada operación.
- El código debe ser legible y ejecutable. Valide los casos de estructuras vacías.
- Utilice únicamente los recursos autorizados por el docente y prepárese para explicar individualmente su solución.
- Puntaje total: 10 puntos. Parte I: 60 %. Parte II: 40% puntos.

PARTE I. Preguntas de análisis y razonamiento (60%)

1. Selección de estructuras de datos (10 puntos)

Una empresa de comercio electrónico necesita resolver tres situaciones:

A. Procesar solicitudes de despacho en el mismo orden en que fueron recibidas.
B. Permitir que un supervisor revierta la última modificación registrada, cuando esa sea la regla de negocio.
C. Mantener códigos numéricos de productos en un árbol binario de búsqueda para facilitar su consulta.

Responda:

1. Seleccione la estructura de datos apropiada para cada situación.
2. Justifique cada selección según su comportamiento.
3. Explique qué problema operativo surgiría si se utilizara una pila para procesar las solicitudes del apartado A.

1. Selección de la estructura apropiada para cada situación

- **Situación A.** Cola (Queue) con FIFO (First In, First Out).
- **Situación B.** Pila (Stack) con LIFO (Last In, First Out).
- **Situación C.** Árbol Binario de Búsqueda (ABB / BST)

2. Justificación técnica de cada selección

- **Situación A (Cola FIFO).** Un proceso de atención o despacho equitativo requiere que el primer elemento en registrarse sea el primero en procesarse. La estructura de cola garantiza este orden de llegada cronológico, encolando al final y desencolando al frente.
- **Situación B (Pila LIFO).** La acción de revertir o deshacer (undo) requiere acceder de forma inmediata a la operación más reciente. En una pila, el último elemento insertado es el primero en retirarse desde la cima, lo que permite deshacer cambios en orden inverso al que se aplicaron.
- **Situación C (Árbol Binario de Búsqueda).** El árbol binario de búsqueda organiza las claves numéricas de forma jerárquica (valores menos a la izquierda y mayores a la derecha). Esto permite realizar búsquedas, inserciones y validaciones eficientes con un tiempo promedio de complejidad O(log n), superando la búsqueda secuencial lineal O(n) de una lista desordenada.

3. Problema operativo de usar una pila en el apartado A

Si se utilizara una pila (LIFO), se despacharían primero las solicitudes más recientes, dejando las primeras solicitudes recibidas atrapadas en el fondo.

Esto provocaría el fenómeno de inanición (starvation) de solicitudes antiguas: si entran pedidos constantemente, los primeros clientes nunca serían atendidos o sufrirán demoras inaceptables, violando lso acuerdos de nivel de servicio (SLA) y generando insatisfacción operativa en el cliente.

Comisión de Acreditación de la Calidad Académica
UNIVERSIDAD DE ORIENTE - UNIVO
ACREDITADA
2022 - 2027

2. Un sistema de auditoría ejecuta estas operaciones sobre una pila inicialmente vacía:

```
1   pila = []
2   pila.append("M101")
3   pila.append("M102")
4   pila.append("M103")
5   pila.pop()
6   pila.append("M104")
7   pila.pop()
```

1. Represente el contenido de la pila después de cada operación.
2. Indique el elemento que queda en la cima al finalizar.
3. Escriba el contenido final desde la cima hasta la base.
4. Explique por qué pop(0) no representa la retirada del último elemento de una pila convencional implementada con listas.

1. Contenido de la pila después de cada operación

Operación	Contenido de la pila (base a cima)
Inicial	**[]**
append M101	**[“M101”]**
append M102	**[“M101”, “M102”]**
append M103	**[“M101”, “M102”, “M103”]**
pop()	**[“M101”, “M102”]**
append M104	**[“M101”, “M102”, “M104”]**
pop()	**[“M101”, “M102”]**

2. Elemento que queda en la cima al finalizar

El elemento en la cima es “M102” (el último índice de la lista)

3. Contenido final desde la cima hasta la base

Cima -> Base: “M102”, “M101”

4. Justificación de por qué pop(0) no representa la retirada de la cima

En una implementación estándar de pila sobre listas de Python donde se apila con “.append()”, el extremo activo (la cima) se ubica al final de la lista (índice -1).

La instrucción “pop(0)” extrae el elemento ubicado en la posición 0, que corresponde a la base (el elemento más antiguo), operando como una cola (FIFO) y violando la propiedad fundamenta lLIFO. Para desapilar correctamente se debe invocar “pop()” sin argumentos. Adicionalmente, “pop(0)” tiene un costo temporal de O(n) al desplazar todos los elementos, mientras que retirar de la cima es O(1).

Comisión de Acreditación de la Calidad Académica
UNIVERSIDAD DE ORIENTE - UNIVO
ACREDITADA
2022 - 2027

3. Un centro de soporte recibe tickets en una cola inicialmente vacía. Se ejecutan las operaciones:

```
cola = []
cola.append("T01")
cola.append("T02")
cola.append("T03")
cola.pop(0)
cola.append("T04")
cola.pop(0)
cola.append("T05")
```

1. Muestre el contenido de la cola después de cada operación.
2. Determine qué ticket se atendería a continuación y cuál sería el siguiente.
3. Explique la consecuencia de atender siempre el último ticket ingresado.
4. Indique qué condición debe verificarse antes de retirar un elemento.

1. Representación del contenido después de cada operación

Operación	Contenido de la cola (primero a último)
Inicial	**[]**
append T01	**[“T01”]**
append T02	**[“T01”, “T02”]**
append T03	**[“T01”, “T02”, “T03”]**
pop(0)	**[“T02”, “T03”]**
append T04	**[“T02”, “T03”, “T04”]**
pop(0)	**[“T03”, “T04”]**
append T05	**[“T02”, “T03”, “T05”]**

2. Tickets a atender a continuación

- Atendido a continuación: “T03” (es el elemento que se encuentra actualmente en el frente de la cola, índice 0).
- El siguiente después de ese: “T04” (pasará al frente una vez retirado T03)

3. Consecuencia de atender siempre el último ticket ingresado

Al atender siempre el último ingresado se invierte el principio de equidad de uan cola, convirtiéndola en una pila (LIFO).

Por lo que cause retraso indefinido e inanición (starvation) de los tickets iniciales; los clientes que reportaron sus incidencias con mayor antelación quedarían desatendidos mientras sigan ingresando casos nuevos al centro de soporte.

4. Condición que debe verificarse antes de retirar un elemento

Debe verificarse que la cola no esté vacía (por ejemplo: “len(cola) > 0” o “if cola:”). Ya que si se ejecuta pop(0) sobre una lista vacía, el intérprete lanzará una excepción en tiempo de ejecución de tipo “IndexError: pop from empty list” (error de subdesbordamiento o underflow), provocando la detención inesperada del sistema.

4. Árbol binario de búsqueda

Un almacén inserta estos códigos en un árbol binario de búsqueda, en el orden indicado:

50, 30, 70, 20, 40, 60, 80

Dibuje el árbol resultante.

- **50**: es la raíz.
- **30**: menor que 50, va a la izquierda de 50.
- **70**: mayor que 50, va a la derecha de 50.
- **20**: menor que 50, va a la izquierda; menor que 30, va a la izquierda de 30.
- **40**: menor que 50, va a la izquierda; mayor que 30, va a la derecha de 30.
- **60**: mayor que 50, va a la derecha; menor que 70, va a la izquierda de 70.
- **80**: mayor que 50, va a la derecha; mayor que 70, va a la derecha de 70.

Explique cómo determinar si el código 60 existe.

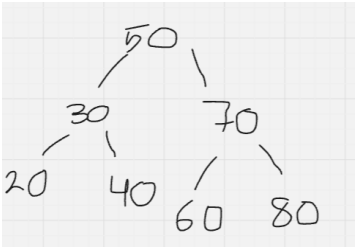
Se hace una búsqueda binaria en el árbol, comenzando por la raíz:

1. Comparar 60 con **50**: 60 > 50, se va al subárbol derecho.
2. Comparar 60 con **70**: 60 < 70, se va al subárbol izquierdo.
3. Comparar 60 con **60**: son iguales, el código existe.

Justifique por qué el recorrido inorden presenta los códigos ordenados de menor a mayor.

R// Si en algún momento se llegara a un puntero vacío (None) sin encontrarlo, el código no existiría. En este caso bastaron 3 comparaciones, porque en cada paso se descarta la mitad del árbol.

Dibujo del árbol:



Comisión de Acreditación de la Calidad Académica
UNIVERSIDAD DE ORIENTE - UNIVO
ACREDITADA
2022 - 2027

Ahora muestre cual será el resultado en Inorden:

El recorrido inorden sigue el orden: izquierda → raíz → derecha.

20, 30, 40, 50, 60, 70, 80

Explicación:

En un árbol binario de búsqueda se cumple, para cada nodo, que:

- Todos los valores del subárbol **izquierdo** son **menores** que el nodo.
- Todos los valores del subárbol **derecho** son **mayores** que el nodo.

El recorrido inorden visita primero el subárbol izquierdo (valores menores), luego el nodo, y al final el subárbol derecho (valores mayores). Como esto se repite de forma recursiva en cada subárbol, los valores se visitan siempre de menor a mayor, lo que produce una secuencia ordenada de forma ascendente.

5. Eliminación en un árbol (10 puntos)

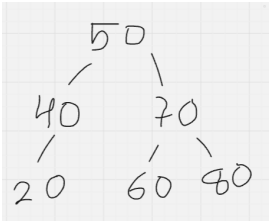
En el árbol de la pregunta anterior se necesita eliminar el código 30.

1. Identifique cuántos hijos tiene el nodo que se eliminará.
R// El nodo 30 tiene 2 hijos: el 20 (izquierdo) y el 40 (derecho)

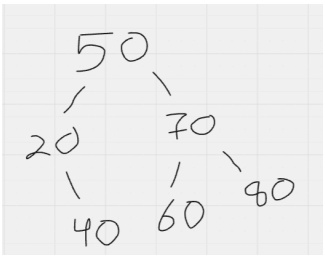
2. Explique el procedimiento para eliminarlo sin romper las propiedades del árbol binario de búsqueda.
R// Cuando el nodo tiene dos hijos, no se puede simplemente quitar, porque quedarían dos subárboles sin padre. El procedimiento es:

1. Buscar el nodo 30 recorriendo el árbol desde la raíz.
2. Como tiene dos hijos, elegir un sucesor que lo reemplace. Hay dos opciones válidas:
    - Sucesor inorden: el menor valor del subárbol derecho. En este caso, el subárbol derecho de 30 es solo el 40, así que el sucesor es 40.
    - Predecesor inorden: el mayor valor del subárbol izquierdo (en este caso, el 20).
3. Copiar el valor del sucesor (40) en el nodo que se elimina (30).
4. Eliminar el nodo original del sucesor (el 40 de abajo), que al ser una hoja se quita directamente.

3. Dibuje el árbol después de la eliminación.
R//
Usando el sucesor inorder:



Con un predecesor:



Comisión de Acreditación de la Calidad Académica
UNIVERSIDAD DE ORIENTE - UNIVO
ACREDITADA
2022 - 2027

4.Indique qué propiedad se podría incumplir si se reemplazara el nodo 30 por cualquier código sin verificar su relación con los demás nodos.

R//Se podría violar la propiedad fundamental del árbol binario de búsqueda: para cada nodo, todos los valores del subárbol izquierdo deben ser menores y todos los del subárbol derecho deben ser mayores.

Por ejemplo, si se reemplaza el 30 por 45, el 45 quedaría en el subárbol izquierdo de 50 (lo cual está bien), pero tendría el 40 como hijo derecho y 40 < 45, es decir, un valor menor en el subárbol derecho. Se rompería el orden y la búsqueda dejaría de funcionar correctamente: al buscar el 40, se comparar con 45, se iría a la izquierda y nunca se encontraría, aunque exista. Además, el recorrido inorden dejaría de salir ordenado.

Desarrollo y árbol resultante:

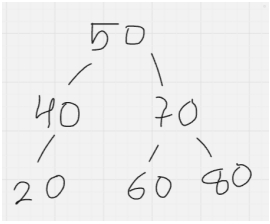
Nodo a eliminar: 30

Se busca el 30 desde la raiz: 30 < 50, se va a la izquierda y se encuentra

Como tiene 2 hijos, se reemplaza por su sucesor inorden el cual seria el 40

Se copia el 40 en el nodo 30

Se elimina el nodo 40 original



6. Diagnóstico de una solución: Un desarrollador propone el siguiente código para atender solicitudes:

```
1   solicitudes = ["A01", "A02", "A03"]
2
3   atendida = solicitudes.pop()
4   print("Solicitud atendida:", atendida)
```

El requisito empresarial establece que las solicitudes deben atenderse por orden de llegada.

1. **¿Qué solicitud se atiende con el código actual?**
   R// Se atiende **"A03"**, porque `pop()` sin argumentos elimina y devuelve el último elemento de la lista. La salida es `Solicitud atendida: A03`.
2. **¿Qué estructura y operación se están representando realmente?**
   R// Una pila (stack), que funciona con LIFO . El `pop()` es la operación de sacar el tope de la pila.
3. **Corrija la instrucción para cumplir el requisito usando una lista de Python.**
   R// El requisito pide orden de llegada (FIFO), que corresponde a una cola (queue). Se debe sacar el primer elemento de la lista

4. **Explique por qué el programa puede ejecutarse sin errores y aun así incumplir la regla de negocio.**
   R// Porque pop() es una instrucción válida en python, por lo que no da error. Este saca el último elemento (A03) en lugar del primero (A01), y el negocio pide que se atiende por orden de llegada. Habia un error de lógica no de sintaxis

ejercicio.py
```
1   solicitudes = ["A01", "A02", "A03"]
2
3   atendida = solicitudes.pop(0)
4   print("Solicitud atendida:", atendida)
```

PARTE II. Ejercicio práctico integrador (40%)

Caso empresarial: Sistema de gestión de pedidos de una distribuidora

Una distribuidora de productos tecnológicos “Las naranjas” necesita un prototipo en Python para gestionar las solicitudes de sus clientes, controlar las últimas operaciones de despacho y consultar productos por su código. Actualmente, las tareas manuales ocasionan errores en el orden de atención, dificultad para revisar operaciones recientes y demoras al localizar productos.

Desarrolle un programa de consola que utilice una cola FIFO, una pila LIFO y un árbol binario de búsqueda.

A. Gestión de solicitudes mediante FIFO (10 puntos)

1. Registrar solicitudes con un código y el nombre del cliente.(Nombre de los estudiantes)
2. Agregar cada solicitud al final de la cola.
3. Atender la siguiente solicitud respetando el orden de llegada.
4. Mostrar las solicitudes pendientes.
5. Controlar cuando no existan solicitudes para atender.

B. Registro de despachos mediante LIFO (8 puntos)

6. Registrar un despacho con código de operación y código de producto.
7. Agregar cada despacho a la pila.
8. Consultar los despachos registrados.
9. Retirar el último despacho registrado mediante desapilado.
10. Controlar el intento de retirar elementos cuando la pila está vacía.

Nota: retirar un despacho representa una corrección del registro del prototipo; no implica que un producto físicamente despachado regrese automáticamente al almacén.

C. Consulta de productos mediante árbol binario de búsqueda (12 puntos)

11. Crear un árbol binario de búsqueda a partir de códigos numéricos de productos.
12. Implementar la inserción respetando la propiedad de orden del árbol.
13. Buscar un código e informar si existe.
14. Mostrar los códigos mediante un recorrido inorden.
15. Permitir la eliminación de un código, conservando las propiedades del árbol.

D. Integración, validaciones y demostración (10 puntos)

16. Integrar las funcionalidades en un menú de consola.
17. Validar entradas y opciones incorrectas.
18. Demostrar que las tres estructuras funcionan de manera independiente y coordinada dentro del programa.
19. Presentar pruebas que permitan verificar los resultados.

Datos iniciales obligatorios

Estructura	Datos iniciales
Cola FIFO	Solicitudes P001, P002, P003, registradas en ese orden, con su datos como estudiantes.
Pila LIFO	Despachos D001, D002, D003, registrados en ese orden.
Árbol binario de búsqueda	Códigos 50, 30, 70, 20, 40, 60, 80, insertados en ese orden.

Entregables

- Archivo Python ejecutable (.py).
- Document en pdf de las preguntas consultadas con sus nombres.
- Evidencias de las pruebas solicitadas.
- Explicación breve de por qué se utiliza cada estructura.
- Demostración individual/parejas del funcionamiento del programa.